

PSD DE SEÑALES ALEATORIAS (GNURADIO)

Universidad Industrial de Santander
Didier Manuel Correa Gómez, 2212254,
didier2212254@correo.uis.edu.co

Oscar Daniel Castellanos Mariño, 2205024,
oscar2205024@correo.uis.edu.co

[Repositorio de Github](#)

Resumen

¹ La tercera práctica de laboratorio se centra en el estudio de la densidad espectral de potencia (PSD) de diferentes tipos de señales utilizando GNU Radio. El análisis incluye señales binarias bipolares, ruido blanco y datos reales capturados por cámaras y micrófonos, con el objetivo de caracterizar su comportamiento espectral. Para ello, se implementaron flujos de trabajo en GNU Radio que incorporan bloques de interpolación, fuentes virtuales y funciones personalizadas, además de variar parámetros como las muestras por símbolo (SPS). Asimismo, se empleó GitHub como herramienta de apoyo en el desarrollo colaborativo y en la gestión de versiones de los experimentos. Los resultados obtenidos evidencian cómo la eficiencia espectral y el ancho de banda de las señales están directamente relacionados con los esquemas de modulación, resaltando así la importancia práctica del análisis de PSD en el ámbito de las comunicaciones..

Abstract

The third lab focuses on studying the power spectral density (PSD) of different types of signals using GNU Radio. The analysis includes bipolar binary signals, white noise, and real data captured by cameras and microphones, with the aim of characterizing their spectral behavior. To this end, workflows were implemented in GNU Radio that incorporate interpolation blocks, virtual sources, and custom functions, in addition to varying parameters such as samples per symbol (SPS). GitHub was also used as a support tool for collaborative development and version management of the experiments. The results obtained demonstrate how the spectral efficiency and bandwidth of signals are directly related to modulation schemes, thus highlighting the practical importance of PSD analysis in the field of communications.

I. Introducción.

El análisis espectral es una herramienta esencial en el estudio de las comunicaciones, ya que permite comprender cómo se distribuye la energía de una señal en el dominio de la frecuencia. Entre las métricas más relevantes se encuentra la Densidad Espectral de Potencia (PSD, Power Spectral Density), la cual posibilita identificar patrones ocultos en señales, optimizar el uso del espectro disponible y evaluar el impacto del ruido o de las interferencias en los sistemas de transmisión.

En el campo de las telecomunicaciones, la PSD desempeña un papel crucial en el diseño de esquemas de modulación, en la caracterización de canales de comunicación y en la gestión eficiente del ancho de banda. Tecnologías actuales como la radio definida por software (SDR) y las redes de quinta generación (5G) dependen de este tipo de análisis para garantizar transmisiones de alta calidad, robustas frente a entornos hostiles y con capacidad de adaptación a diversas condiciones espectrales.

La práctica desarrollada en este informe se centra en la exploración de la PSD aplicada a señales aleatorias, abarcando tanto señales binarias generadas sintéticamente como señales provenientes de fuentes reales (archivos de imagen y audio). Se busca observar cómo la variación de parámetros, como los *Samples per Symbol (Sps)*, y la implementación de distintas codificaciones (RZ, Manchester) o modulaciones digitales (OOK, BPSK, ASK), afectan la forma espectral de las señales.

De esta manera, el laboratorio no solo permite afianzar los conceptos teóricos de señales aleatorias y densidad espectral, sino también aplicar dichos fundamentos a escenarios prácticos mediante el uso de GNU Radio, fortaleciendo la relación entre teoría y aplicación en el diseño de sistemas de comunicación modernos.

II. Metodología Experimental.

En cuanto al procedimiento realizado para la ejecución de la primera práctica de laboratorio se procedió como sigue:

A. Creación de los Directorios:

Para el desarrollo del laboratorio se inició con la creación de un entorno de trabajo ordenado. Se generaron directorios específicos para almacenar los flujogramas, resultados y avances del informe. Posteriormente, cada integrante del grupo abrió una rama en el repositorio compartido, lo que permitió documentar los aportes individuales y facilitar la integración final del trabajo mediante control de versiones.

B. Implementación de los bloques en GNURadio y Agregando una señal de tipo vector

Como primer escenario de prueba se diseñó un flujograma en GNU Radio capaz de generar señales binarias bipolares. Se modificaron parámetros clave, como la cantidad de muestras por símbolo (*Samples per Symbol, Sps*), con el fin de evaluar su efecto tanto en el dominio temporal como en la densidad espectral de potencia (PSD). En este proceso se combinaron bloques básicos y funciones implementadas directamente por los estudiantes.

C. Análisis de fuentes provenientes del entorno real

Una vez comprendido el comportamiento de las señales artificiales, se procedió a sustituir el generador aleatorio por archivos provenientes del entorno real, tales como una imagen

en formato .jpg y un archivo de audio en formato .wav. Se implementaron bloques para la lectura de datos y conversión de bits, de manera que fuera posible contrastar las diferencias espectrales entre fuentes sintéticas y reales.

D. Configuración de esquemas de codificación y modulación

El siguiente paso consistió en modificar pulsos y filtros con el objetivo de reproducir distintos esquemas de comunicación. Entre ellos se incluyeron codificaciones de línea (como RZ y Manchester) y técnicas de modulación digital (como OOK y BPSK). Estas configuraciones facilitaron el estudio de la influencia que tienen las transformaciones de la señal sobre la distribución espectral y la eficiencia en el uso del ancho de banda.

Finalmente, se respondieron las preguntas de control propuestas en la guía, enfocadas en comprender el rol de bloques específicos, interpretar correctamente las gráficas de PSD y comparar el comportamiento de señales unipolares y bipolares. Los resultados se validaron mediante observaciones directas en GNU Radio y se discutieron en función de los fundamentos teóricos estudiados.

III. Análisis de resultados.

Se realizó un análisis de los bloques presentes en la cadena de procesamiento de señales con el fin de comprender su funcionamiento y evaluar el impacto que tenían en las etapas posteriores. Este estudio incluyó la identificación de parámetros relevantes como la tasa de interpolación, la rata de bits, la frecuencia de muestreo y el ancho de banda, considerando variaciones del parámetro Sps en señales binarias aleatorias bipolares. De esta manera, se observó su comportamiento tanto en el dominio temporal como en la forma de la PSD, así como en los parámetros principales mencionados. El parámetro Sps corresponde al número de muestras por símbolo, es decir, la cantidad de muestras digitales que representan un símbolo, mientras que h corresponde al coeficiente del filtro interpolador. En el caso de $Sps = 4$

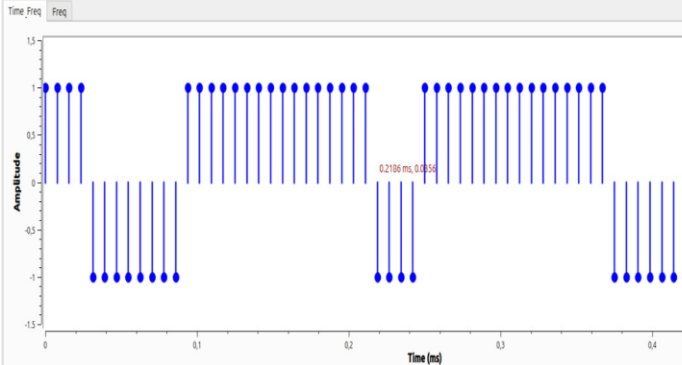


Figura 1. tiempo4

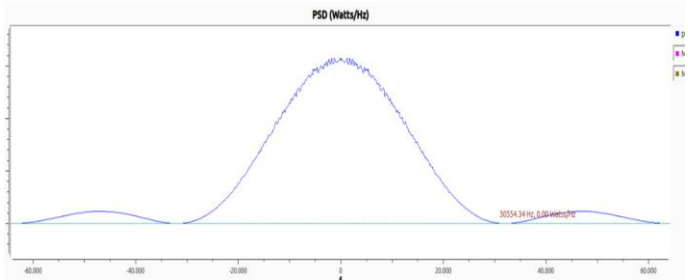


Figura 2. PSD4

En este caso, la frecuencia de muestreo estuvo fijada por el flujograma en 128 kHz. La rata de bits se calculó a partir del inverso del período de bit (TbT_bTb), el cual se pudo identificar en la PSD o, alternativamente, como la diferencia entre la última y la primera muestra dentro de las muestras por símbolo en el dominio del tiempo. Para este escenario, el cálculo fue:

$$Rb = \frac{1}{Tb} = \frac{1}{30554.32}$$

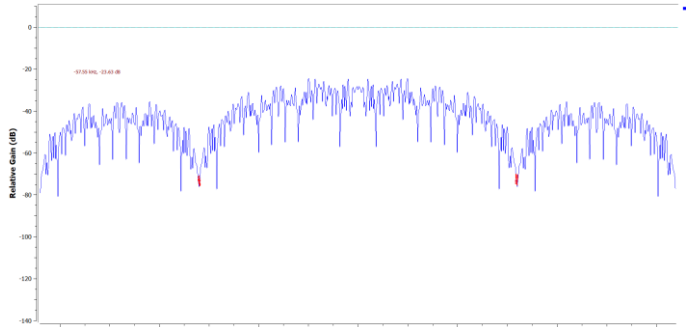


Figura 3. ferq4

Para determinar el ancho de banda, se ubicaron los cursores en los puntos de transición de los lóbulos observados en el dominio de la frecuencia, específicamente en las zonas resaltadas en rojo. Al calcular la diferencia entre dichas frecuencias, el ancho de banda obtenido fue de 65.15 kHz, teniendo como uno de sus límites 32.57 kHz. Este resultado correspondió al caso de $Sps=8Sps=8Sps=8$.

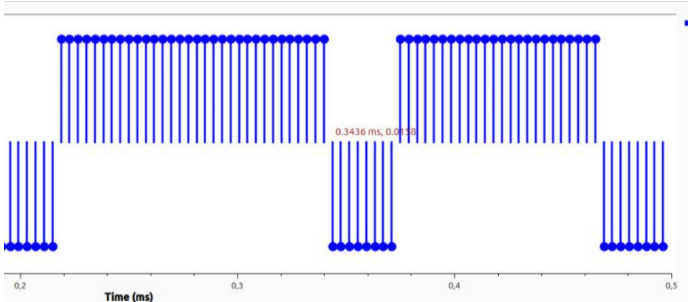


Figura 4. tiempo8

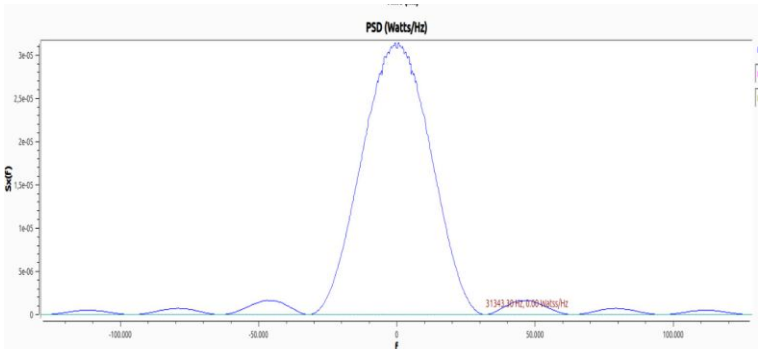


Figura 5. PSD8

En este caso, se aplicó el mismo procedimiento para calcular los parámetros solicitados. Al aumentar las muestras por símbolo a Sps=8, se observó lo esperado: el pulso en el dominio del tiempo se hizo más ancho y, en consecuencia, la PSD se volvió más angosta en frecuencia. Aunque el ancho de banda se mantuvo, en el espectro los lóbulos se repitieron una vez más en comparación con el escenario de Sps=4.

El experimento también se realizó con Sps=16 y Sps=1. Para Sps=16, se presentó el mismo comportamiento descrito: un mayor número de muestras por símbolo redujo la dispersión espectral, haciendo la PSD más angosta sin alterar el ancho de banda. En contraste, con Sps=1, la PSD se volvió prácticamente plana, ocupando la totalidad del espectro disponible. Esto ocurrió porque al haber únicamente una muestra por símbolo no existió una forma de pulso que limitara el ancho de banda, lo que generó una distribución espectral más amplia.

Comprobar cómo es el ruido blanco en tiempo y en PSD:
se configuró el flujograma de la manera descrita por el documento de la práctica. y se obtuvieron los siguientes resultados con sps=1

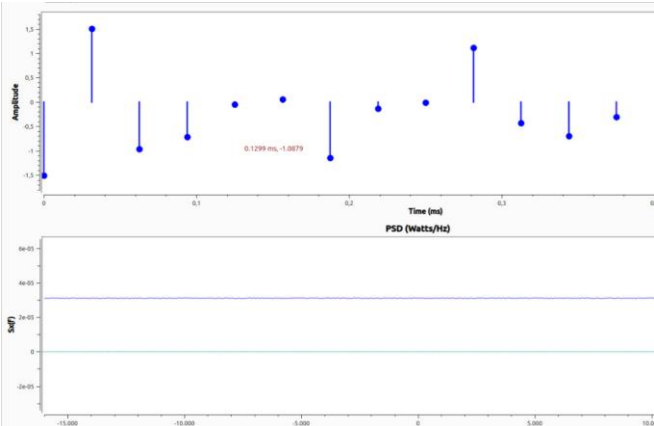


Figura 6. ruido4

Se observó que, con Sps=1, las muestras por símbolo no formaron una secuencia definida como en los casos anteriores de señales binarias, sino que presentaron un comportamiento completamente aleatorio. En este escenario, la PSD resultó plana o muy ancha, similar a la de una señal de ruido blanco, ya que la potencia se distribuyó de manera uniforme en todas las frecuencias.

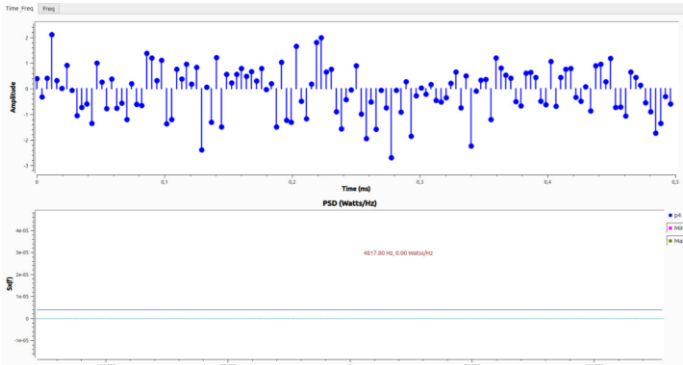


Figura 7. ruido8

Con Sps=8 se observó el incremento en el número de muestras por símbolo, tal como se esperaba. Sin embargo, la PSD se mantuvo plana, ya que la señal correspondió a un proceso de ruido blanco.

Bits provenientes de una fuente real:

En este punto se utilizó la imagen recomendada en la guía, correspondiente a la rana. El archivo fue cargado y posteriormente se visualizó su PSD.

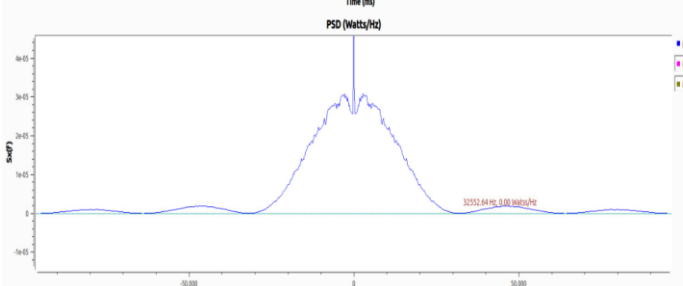


Figura 8. PSDrana

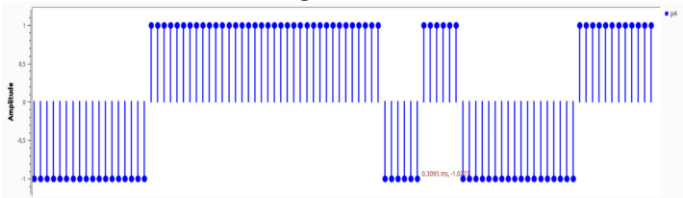


Figura 9. tiempo-rana

Se observó que la PSD presentó la forma clásica, con algunos picos alrededor de la frecuencia central, donde se concentró la mayor potencia. Esto ocurrió porque en la imagen utilizada predominaba el color verde; por lo tanto, en las bajas frecuencias aparecieron picos de potencia asociados a dicho color. Si la imagen hubiera tenido otro patrón con una fuerte presencia de un color diferente, habrían surgido nuevos picos similares en frecuencias cercanas, representando esa tonalidad. En el dominio del tiempo, el comportamiento de la señal resultó similar al del primer ejemplo de la señal binaria bipolar.

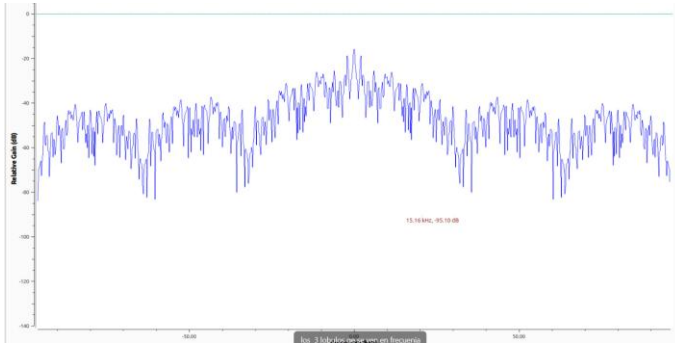


Figura 10. freq-rana

En el dominio de la frecuencia se identificó el ancho de banda y la representación de los lóbulos, mostrando un comportamiento muy similar al de la señal binaria bipolar del primer ejemplo, con un ancho de banda equivalente. En esta parte de la práctica se utilizó el audio recomendado en la guía, el cual correspondía a un ejercicio de *listening* en inglés. El archivo fue cargado de manera análoga al procedimiento realizado con la imagen de la rana y se evidenció tanto su comportamiento en el tiempo como su PSD.

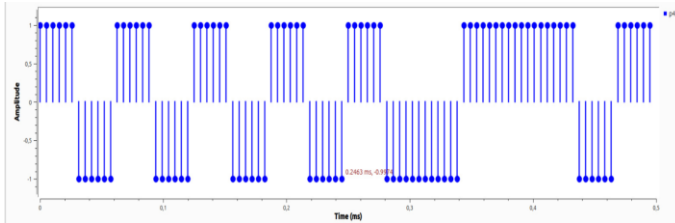


Figura 11 . tiempo-audio

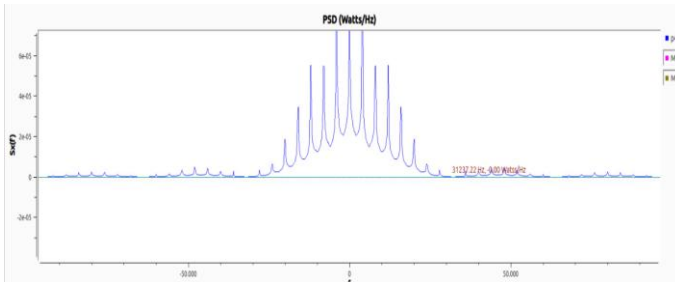


Figura 12 . PSD-audio

Se observó que la señal de audio en el dominio del tiempo presentó un comportamiento similar al de una señal binaria bipolar, mostrando cierta tendencia a formar patrones, aunque no de manera estricta. Esto se debió a que, al tratarse de una grabación de voz humana, su contenido espectral se encontró principalmente en el rango de 0 a 20 kHz, que corresponde a las frecuencias audibles para el oído humano. Esta característica se apreció con mayor claridad en la PSD, donde la potencia se distribuyó dentro de dicho rango y se evidenciaron picos en frecuencias específicas.

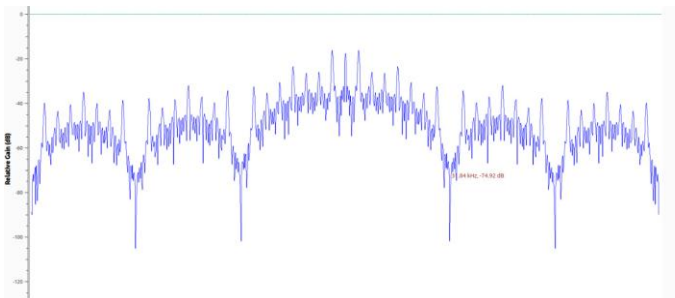


Figura 13 . Freq-audio

En el dominio de la frecuencia se observó que el ancho de banda se mantuvo igual al del ejemplo de la rana y que la cantidad de lóbulos fue la misma.

IV. Preguntas de Control

A. ¿Qué papel juega la siguiente combinación de bloques Figura 13?



Figura 13 . Combinación de bloques de GnuRadio

La combinación de bloques se utilizó para añadir un nivel DC a la señal, lo que permitió convertirla en una forma bipolar. Posteriormente, el bloque *Multiply Const* ajustó la amplitud para conservar los valores originales, evitando la atenuación que se habría generado tras el desplazamiento de continua.

B. ¿Qué papel juega el bloque “Interpolation FIR Filter”, ¿cómo funciona?

El bloque *Interpolation FIR Filter* interpola la señal insertando muestras adicionales según el filtro. El parámetro *Interpolation* vale *Sps* porque representa las muestras por símbolo; si se cambia, la señal se distorsiona. Para analizar la señal en *p3* se conecta la instrumentación antes de la interpolación. El ancho de banda en *p4* se obtiene con $B = \frac{Rb * Sps}{2}$. La frecuencia de

muestreo en *p3* se calcula como $f_{sp3} = \frac{f_{sp4}}{Sps}$.

C. ¿Por qué razón la PSD de las señales binarias que provienen de una señal de audio es diferente a la que proviene de una foto siendo ellas igualmente señales binarias bipolares de forma rectangular?

La PSD de señales binarias derivadas de audio se concentra en ciertas bandas de frecuencia, ya que los sonidos, como la voz, ocupan rangos específicos. En cambio, en las imágenes la información se distribuye de manera más aleatoria entre píxeles, lo que genera una PSD más uniforme y sin concentraciones marcadas.

D. ¿Qué papel juega el bloque “Throttle”?

El bloque *Throttle* actúa como un limitador de velocidad dentro del esquema. Su función principal es fijar la tasa de muestreo a un valor determinado, de modo que el flujo de datos no exceda la capacidad del procesador. Gracias a esto, la simulación se mantiene estable y evita que el sistema se sature al manejar señales en tiempo real.

E. ¿Qué pasaría con la PSD si no se hace la conversión a señal bipolar, sino que la señal binaria en *p4* solo tiene valores de 0 ó 1 en lugar de -1 ó 1?

Si la señal permanece en formato unipolar (0 y 1) aparece un componente de continua en la PSD, lo que genera un pico en la frecuencia cero debido a que la media de la señal no es nula. En cambio, cuando se usa una señal bipolar (-1 y +1), dicho

componente desaparece y la energía se reparte de forma simétrica alrededor del espectro. Aunque la forma general del espectro se conserva, en la versión unipolar se añade esa distorsión asociada al nivel DC.

- F. *Se supone que el ruido blanco tiene un ancho de banda infinito, ¿coincide esto con lo observado en GNU Radio?, ¿por qué?*

En teoría, el ruido blanco posee una PSD constante en todas las frecuencias, lo que implica un ancho de banda infinito. Sin embargo, en GNU Radio esto no se cumple, ya que la simulación depende de la frecuencia de muestreo. Por esa razón, la PSD se observa plana únicamente dentro del rango comprendido entre $\pm \frac{f_s}{2}$.

- G. *Se supone que una señal binaria aleatoria de forma rectangular tiene un ancho de banda infinito, ¿coincide esto con lo observado en GNU Radio y por qué?*

Una señal rectangular ideal debería mostrar un ancho de banda infinito. Sin embargo, en GNU Radio se aprecia un espectro limitado, donde los lóbulos de la función tipo *sinc* dependen directamente de los valores de Sps. A medida que aumenta el número de muestras por símbolo, los lóbulos aparecen más próximos, lo que indica una reducción en el ancho de banda y evidencia la relación inversa entre duración del símbolo y extensión espectral.

- H. *¿Qué fórmula podría ayudar a calcular el número de lóbulos de la PSD de señal binaria aleatoria de forma rectangular cuando se conoce la frecuencia de muestreo y Sps? Nota: el lóbulo de la mitad se cuenta como dos porque tiene el doble de ancho que los demás.*

El cálculo del número de lóbulos en la PSD puede hacerse relacionando el ancho de banda con la frecuencia de la señal. Dado que cada lóbulo ocupa un espacio proporcional a la frecuencia de muestreo, la estimación resulta en: $N_{\text{simbolos}} = \frac{Sps}{2}$, Como el lóbulo central tiene el doble de anchura que los demás, este se contabiliza como dos, lo que finalmente lleva a que el número total de lóbulos sea aproximadamente igual a Sps.

- I. *¿Cómo se calcula todo el rango de frecuencias que ocupa el espectro cuando se conoce Rb y Sps?*

El rango de frecuencias se determina a partir de la tasa de bits y las muestras por símbolo. De manera aproximada, el ancho de banda se expresa como $B = \frac{Sps}{2} * Rb$, Así, al incrementar Sps, el tiempo de símbolo se alarga y el espectro ocupado se reduce.

- J. *¿Cómo se calcula la resolución espectral del analizador de espectros, cuando se conoce N y la frecuencia de muestreo?*

La resolución espectral se obtiene dividiendo la frecuencia de muestreo entre el número de puntos de la FFT: $\Delta f = \frac{f_s}{N}$, Un

mayor valor de N produce una resolución más fina, lo que facilita separar componentes de frecuencia muy próximas.

- K. *¿Qué pasaría si en el bloque “Unpack K Bits” se configura el parámetro K como 16?*

Si en el bloque *Unpack K Bits* se fija K=16, cada muestra se divide en un conjunto de 16 bits. Esto provoca secuencias más extensas de ceros y unos, lo que introduce patrones repetitivos en la señal. Como consecuencia, aparecen armónicos adicionales en el espectro y se incrementa la componente de continua en la PSD

- L. *¿Cómo calcularía la frecuencia de muestreo a la entrada del bloque “Unpack K Bits” si conoce el número de lóbulos de la PSD y el ancho de banda de la señal?*

La frecuencia de muestreo a la entrada del bloque *Unpack K Bits* se puede obtener relacionando el número de lóbulos de la PSD con la tasa de símbolos. Cada lóbulo está asociado a la velocidad de transmisión, por lo que la expresión aproximada es: $f_s = N_{\text{lobulos}} * R_{\text{simbolos}}$. De esta forma, se asegura que la frecuencia de muestreo sea al menos el doble del ancho de banda de la señal, cumpliendo con el criterio de Nyquist.

- M. *¿Cómo calcularía la frecuencia de muestreo a la salida del bloque “Unpack K Bits” si conoce la frecuencia de muestreo a la entrada?*

El bloque *Unpack K Bits* transforma cada byte en K bits. Por esta razón, la frecuencia de muestreo a la salida se calcula como: $f_{\text{salida}} = K * f_{\text{entrada}}$, Esto ocurre porque de cada muestra de entrada se obtienen K muestras en la salida.

- N. *¿Cómo calcularía la frecuencia de muestreo a la salida del bloque “Char to Float” si conoce la frecuencia de muestreo a la entrada?*

El bloque *Char to Float* no modifica la tasa de muestreo, únicamente convierte el formato de los datos. Por lo tanto: $f_{\text{salida}} = f_{\text{entrada}}$, La frecuencia de muestreo permanece igual en la entrada y en la salida.

- O. *¿Para qué caso de Sps la PSD de una señal binaria aleatoria bipolar es similar a la PSD de ruido blanco?*

Cuando Sps=1, la PSD de una señal binaria bipolar se aproxima al comportamiento del ruido blanco. Esto ocurre porque cada símbolo es independiente y tiene la misma probabilidad, lo que produce un espectro prácticamente plano sin lóbulos predominantes.

- P. *¿Qué cambios mínimos haría al flujograma, manipulando principalmente h, si desea que los bits en la señal binaria aleatoria tomen la forma de dientes de sierra?*

Para obtener pulsos con forma de diente de sierra, se ajusta el vector h con dos coeficientes. De esta manera, la señal presenta una transición lineal entre símbolos consecutivos, generando el patrón deseado.

- Q. ¿Qué cambios mínimos haría al flujograma, manipulando principalmente h , si desea que la señal binaria aleatoria tenga codificación de línea Unipolar RZ?

Para implementar la codificación Unipolar RZ, el vector h se define con una parte activa seguida de ceros. Con este ajuste, cada pulso ocupa solo la mitad del intervalo del símbolo y luego retorna a cero, logrando la forma característica de este tipo de codificación.

- R. ¿Qué cambios mínimos haría al flujograma, manipulando principalmente h , si desea que la señal binaria aleatoria tenga codificación de línea Manchester NRZ?

Para generar la codificación Manchester NRZ, el vector h se configura de modo que la primera mitad del símbolo tenga valor positivo y la segunda mitad valor negativo. Este cambio provoca una inversión dentro de cada intervalo, asegurando la transición central que caracteriza esta técnica y facilitando la sincronización.

- S. ¿Qué cambios mínimos haría en el flujograma, aprovechando h y el FIR Interpolating Filter para que la señal binaria tenga la forma de señal OOK?

Para obtener esta modulación, el vector h se define como una senoide ($\sin(\frac{2\pi}{T_0}(15n))$) y la señal se multiplica por dicha portadora. Además, el bloque *Constant Source* se ajusta en cero para evitar un desplazamiento de continua.

- T. ¿Qué cambios mínimos haría en el flujograma, aprovechando h y el FIR Interpolating Filter para que la señal binaria tenga la forma de señal BPSK?

Se utiliza el mismo vector sinusoidal que en OOK, pero antes de la multiplicación la señal binaria se convierte a bipolar ($\pm A$). Con esto, los símbolos producen un cambio de fase de 180° , que caracteriza a la modulación BPSK.

- U. ¿Qué cambios mínimos haría en el flujograma, aprovechando h y el FIR Interpolating Filter para que la señal binaria tenga la forma de señal ASK?

En este caso también se emplea un h sinusoidal, pero la señal se escala a una fracción de su amplitud. Esto genera variaciones de amplitud entre símbolos, propias de la modulación ASK.

- V. ¿Qué cambios mínimos haría en el flujograma, aprovechando h y el FIR Interpolating Filter para que la señal binaria tenga la forma de los latidos del corazón?

Para recrear la forma de un latido cardíaco en la señal binaria, el vector h se diseña con coeficientes no lineales que imitan el patrón característico del pulso. Un ejemplo de esta configuración es:

$$h = [0, 0, -0.25, 0, 0.25, 0, 0, 1, 0, -1, -0.25, 0.5, 0, 0.25, 0, -0.25, 0, 0, 0]$$

- W. ¿Qué cambios mínimos haría en el flujograma, aprovechando h y el FIR Interpolating Filter para que la señal binaria tenga la forma que se muestra en la Figura ?

Para obtener pulsos rizados como los de la figura, el vector h se construye a partir de una función logarítmica, por ejemplo $\ln(\frac{1}{100}x)$. Al variar el valor de x , se genera una envolvente ondulada y progresiva que, aplicada mediante el FIR Interpolating Filter, produce la forma deseada en la señal binaria.

- X. Explique usando gráficas de PSD la diferencia que existe entre la PSD de una señal binaria bipolar y una unipolar.

La PSD de una señal unipolar presenta un componente de continua, visible como un pico en la frecuencia cero, además de lóbulos que no son completamente simétricos. En contraste, la señal bipolar elimina ese componente DC y distribuye la energía de manera equilibrada a lo largo del espectro, mostrando lóbulos simétricos. Esto hace que la versión bipolar utilice el espectro de forma más eficiente en aplicaciones de comunicación.

V. Conclusiones

- Fue importante identificar cuándo una señalera de tipo blanca, ya que esto indicaba que la potencia se distribuía de manera uniforme en todas las frecuencias.
- Los picos de potencia en las bajas frecuencias de la PSD de una señal real se interpretaron como una mayor concentración de energía en esa zona del espectro, lo que reflejaba la presencia de información relevante. En el caso de imágenes, estos picos podían asociarse con tonalidades predominantes, mientras que en audio correspondían a determinados momentos o tonos presentes en esas frecuencias.
- La calidad de los resultados dependió tanto del orden del filtro como de la configuración del vector h . Se reconoció que, al trabajar con un mayor número de muestras en el tiempo, la PSD tendía a hacerse más angosta. En última instancia, todo estuvo condicionado por el objetivo específico que se buscaba alcanzar en el análisis.

Referencias

- [1] chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/<https://lms.>

uis.edu.co/ava/pluginfile.php/535830/mod_resource/content/2/PSD_OT_English.pdf

- [2] https://drive.google.com/file/d/1fd9M4_bIjwLOajQdkdN9ex2pIYRoXomz/view
- [3] GNU Radio. *Python Module - GNU Radio*. Disponible en:
https://wiki.gnuradio.org/index.php?title=Python_Module
- [4] Repositorio: https://github.com/Daniel24-web/CommlI_labB1_G1_DANIEL_DIDIER/tree/main